

SCSS Folder Structure (ITCSS-inspired)

This document defines a clean, scalable `_sass/` folder structure based on **ITCSS (Inverted Triangle CSS)**, adapted for a Jekyll-based personal site.

The goal is:

- predictable source order
- minimal specificity conflicts
- easy theming and long-term maintainability

High-level Structure

```
_sass/  
├── 0-settings/  
├── 1-tools/  
├── 2-generic/  
├── 3-base/  
├── 4-objects/  
├── 5-components/  
├── 6-layouts/  
└── 7-trumps/
```

Each folder represents a *layer* in ITCSS. Files are loaded top → bottom.

0-settings/ — Global configuration (no CSS output)

Purpose

- Global design tokens
- Configuration switches
- Theme definitions

Rules

- Variables only
- No selectors
- No mixins that emit CSS

Typical files

```
0-settings/  
├─ _config.scss          # feature flags, booleans, layout toggles  
├─ _breakpoints.scss     # named media-query values  
├─ _spacing.scss         # spacing scale (4px, 8px, 16px...)  
├─ _typography.scss      # font stacks, type scale  
├─ _palette-*.scss       # raw color palettes  
├─ _colors-*light.scss   # semantic color maps (light)  
└─ _colors-*dark.scss    # semantic color maps (dark)
```

Example

```
$space-3: 0.75rem;  
$bp-md: 48rem;
```

1-tools/ — Sass helpers (no CSS on their own)

Purpose

- Mixins
- Functions
- Sass logic

Rules

- May output CSS *only when included*
- Never define concrete UI

Typical files

```
1-tools/  
├─ _mixins.scss  
├─ _functions.scss  
├─ _grid.scss  
└─ _syntax-highlighting.scss
```

Example

```
@mixin respond($bp) {  
  @media (min-width: map-get($breakpoints, $bp)) {  
    @content;  
  }  
}
```

2-generic/ — Ground-zero styles

Purpose

- Reset browser inconsistencies
- Apply lowest-specificity rules

Rules

- Very broad selectors
- No design decisions

Typical files

```
2-generic/  
├─ _normalize.scss  
├─ _reset.scss  
└─ _box-sizing.scss
```

Example

```
*,
*::before,
*::after {
  box-sizing: border-box;
}
```

3-base/ — Unclassed HTML elements

Purpose

- Default styling for raw HTML
- Typography, links, lists

Rules

- Type selectors only
- No classes

Typical files

```
3-base/  
├─ _base.scss  
├─ _typography.scss  
└─ _links.scss
```

Example

```
h1 {  
  font-size: var(--step-5);  
}
```

4-objects/ — Structural patterns (OOCSS)

Purpose

- Reusable layout patterns
- No cosmetics

Rules

- Class-based
- No colors, no fonts

Typical files

```
4-objects/  
├─ _container.scss  
├─ _media.scss  
└─ _list.scss
```

Example

```
.o-stack > * + * {  
  margin-top: var(--space-4);  
}
```

5-components/ — Designed UI components

Purpose

- Buttons, cards, nav, pagination
- Concrete UI pieces

Rules

- Explicit naming
- Can use colors, fonts, borders

Typical files

```
5-components/  
├─ _buttons.scss  
├─ _card.scss  
├─ _pagination.scss  
└─ _footer.scss
```

Example

```
.c-button {  
  background: var(--color-accent);  
  border-radius: 0.375rem;  
}
```

6-layouts/ — Page-level composition

Purpose

- Styles that only apply in specific layouts
- Home, post, page templates

Rules

- High specificity allowed
- Tied to layout context

Typical files

```
6-layouts/  
├─ _home.scss  
├─ _post.scss  
└─ _page.scss
```

Example

```
.layout-post .c-header {  
  margin-bottom: var(--space-6);  
}
```

7-trumps/ — Utilities & overrides

Purpose

- Last-resort helpers
- One-off overrides

Rules

- !important allowed
- Single responsibility per class

Typical files

```
7-trumps/  
├─ _helpers.scss  
├─ _visibility.scss
```

Example

```
.u-hidden {  
  display: none !important;  
}
```

Import Order (main.scss)

```
@use "0-settings/config";
@use "0-settings/breakpoints";
@use "0-settings/spacing";
@use "0-settings/typography";

@use "1-tools/mixins";
@use "1-tools/functions";

@use "2-generic/normalize";
@use "2-generic/reset";

@use "3-base/base";

@use "4-objects/container";

@use "5-components/buttons";

@use "6-layouts/post";

@use "7-trumps/helpers";
```

Key Principles

- Specificity always increases downward
- Never undo styles — only add
- Each file has *one clear responsibility*
- You should always know **where** a rule belongs

This structure scales cleanly from a small portfolio to a large design system.